

Node.js + React Senior Interview Prep

Real company-reported questions, ranked by priority, with interview-ready answers

How to use this: the **Priority** section covers the 20 questions most repeated across Laerdal, Noccarr, Coditas, Pearson, and Cognizant candidate reports — master these first. Each card has a green "Make it yours" tip — swap in your own numbers, project names, and incidents there. A memorized-sounding answer is the fastest way to lose credibility; a specific number or a real incident is what makes an interviewer believe you actually did the work.

Priority — master these first (20 questions)

Laerdal Medical Corp. | reported Q1

Explain a system you built, a production issue you faced, and how you would scale it.

I'll walk through the architecture and traffic pattern first, then describe one real production issue, its impact, how we investigated it, and what we changed. For scaling, I identify the bottleneck before adding servers — CPU, memory, event-loop lag, API latency, database utilization, connection pools, and downstream dependencies. If the service is stateless, I scale horizontally behind a load balancer; for read-heavy paths I add caching; for expensive async work I move it to background queues. I validate every change with load testing and by watching p95/p99 latency, throughput, and error rate.

Make it yours: Use your SSE-based order-tracking dashboard as the system, and name the actual bottleneck you hit (event-loop lag, DB pool, or memory) with a real before/after number.

Laerdal Medical Corp. | reported Q2

How do you detect a memory leak in Node.js?

First I confirm memory is genuinely leaking and not just rising with traffic — I watch RSS, heapUsed, heapTotal, external memory, and GC behavior. If heapUsed keeps climbing after GC runs, I take two heap snapshots at different points in Chrome DevTools and diff the retained objects. I specifically check unbounded caches, global references, closures, timers, and EventEmitter listeners that were never removed. If RSS grows while the V8 heap stays flat, I look at Buffers, streams, and native/external memory instead. Once I find the retaining reference, I fix it and re-verify under realistic load.

Make it yours: Mention that you reproduced a leak locally with a script that pushed objects into an unbounded array and watched RSS climb — concrete and hard to fake.

Laerdal Medical Corp. | reported Q3

How would you increase API performance?

I measure before I optimize — p50, p95, p99 latency, plus distributed tracing to split the request into Node processing time, DB time, cache time, and downstream-API time. If the database is slow, I look at query plans and indexes. If Node itself is the bottleneck, I profile CPU and hunt for synchronous or CPU-heavy code. If a downstream service is slow, I check its timeout, retry, and circuit-breaker behavior. I fix the actual bottleneck, not the one I assumed, and validate the fix with load testing.

Make it yours: Quote a real p95 number you improved (e.g. "420ms to 140ms") — specific numbers read as lived experience, not a script.

Laerdal Medical Corp. | reported Q4

How would you handle a bulk response?

First question: does the client actually need the whole dataset in one response? For large payloads I prefer pagination, filtering, or streaming over building one giant object in memory. I trim the response to required fields, add compression and caching, and on the frontend avoid rendering thousands of rows at once — virtualization instead. The goal across the stack is less DB work, smaller network payload, lower Node memory, and less browser rendering work.

Make it yours: If you've built an export or report feature, mention the row count where you had to switch from JSON dump to pagination/streaming.

Laerdal Medical Corp. | reported Q5

What happens if `process.nextTick()` is used heavily?

`process.nextTick()` callbacks run before the event loop moves to its next phase. If code keeps scheduling large numbers of `nextTick` callbacks — especially recursively — they can starve the event loop, so timers, I/O callbacks, and incoming requests all get delayed. I avoid using `nextTick` for large recurring work; for CPU-heavy tasks I move the work to Worker Threads or a background process instead.

Make it yours: If you've ever chased a "requests hanging but CPU looks idle" bug, that's the natural story to attach here.

Laerdal Medical Corp. | reported Q6

How do you handle graceful shutdown in Node.js?

On `SIGTERM` I stop accepting new traffic, let in-flight requests finish within a timeout, stop pulling new background jobs, close DB and external connections cleanly, then exit. In Kubernetes this also means readiness checks flip before the pod is killed, so traffic stops reaching the instance before termination. This is what prevents requests from being cut off mid-flight during a deployment.

Make it yours: Mention your actual timeout value and what happens to in-flight SSE connections in your order-tracking dashboard when a pod restarts.

Laerdal Medical Corp. | reported Q7

You have a file-processing feature. How would you design it for performance and scale?

I wouldn't load a large file fully into Node memory — I'd stream it and process incrementally, respecting backpressure. If processing is CPU-intensive, I move it off the API process into Worker Threads or a background worker, fronted by a queue so traffic spikes don't create unlimited concurrent processing. I monitor queue depth, processing time, failure rate, CPU, and memory, and scale the worker pool independently from the API servers.

Make it yours: If you've built any CSV/report upload flow, walk through the actual queue (BullMQ, SQS, etc.) and worker count you used.

Noccarc | reported Q8

You have a memory leak or heap out-of-memory issue. How would you handle it?

I first determine whether it's a genuine JS memory leak or just a legitimate rise in workload — monitoring heap usage, RSS, and GC behavior. Then I capture heap snapshots comparing a healthy state to the high-memory state and look at retained objects: large arrays, caches, closures, EventEmitter listeners, timers. If large files or responses are the cause, I switch to streams and backpressure instead of loading everything into memory. Bumping `--max-old-space-size` can buy time, but I treat that as mitigation, not the fix.

Make it yours: Tie this to the memory-leak demo script you've already run locally — the RSS-climbing numbers you personally saw.

Noccarc | reported Q9

How would you optimize large-list rendering in React?

I profile first with React DevTools to see whether the cost is DOM node count or unnecessary re-renders. For thousands of rows I use virtualization or pagination. I check for stable keys and for props that get new object or function references on every render, which silently defeat memoization. I reach for `React.memo`, `useMemo`, or `useCallback` only after profiling shows they solve a real problem, and for very large datasets I push filtering and pagination to the server instead of shipping everything to the browser.

Make it yours: Name the actual list you optimized (order history, dashboard table) and the row count where virtualization became necessary.

Neuro Spark Works Solutions | reported Q10

How would you optimize a MERN stack application for better performance?

I treat it as one end-to-end problem, not three separate ones. On MongoDB: slow queries, execution plans, indexes, and avoiding unnecessary fields in the response. On Node: profiling slow endpoints, synchronous operations, connection pools, and serialization cost. On React: Profiler to catch unnecessary renders and expensive components, virtualization for large lists. Across the board I check payload size, caching, and

compression, and I always measure latency, throughput, error rate, and frontend metrics before and after the change.

Make it yours: Pick one layer where you have a real number — an index that cut a query from seconds to milliseconds is a strong opener.

G2i | reported Q11

What could cause performance bottlenecks in a React application?

I split the investigation into rendering, JS execution, network, and DOM size. For rendering, React Profiler shows which components render unnecessarily or take real time. I check whether parent state updates cascade into large re-renders, whether props carry freshly-created objects or functions, and whether expensive calculations happen during render instead of being memoized. For large lists, virtualization. I also check the browser's Performance tab for long main-thread tasks and the Network tab for slow APIs or large payloads — then I fix the actual bottleneck instead of reflexively adding memoization everywhere.

Make it yours: Mention a specific case where you initially suspected React re-renders but the real cause turned out to be an API or payload issue — that self-correction reads as senior judgment.

Public 2026 Node.js candidate report | reported Q12

How do you investigate a sudden spike in API latency?

First, when did it start and is it every endpoint or one specific endpoint? I compare p50/p95/p99 against request volume, CPU, memory, and error rate, then use distributed tracing to see whether the extra latency is in Node processing, the database, Redis, or a downstream service. I check recent deployments and config changes. If a deployment caused it and impact is severe, I roll back or disable the feature while I keep investigating the root cause, then verify latency under real traffic afterward and add alerting to catch a repeat.

Make it yours: If you've ever done a rollback under pressure, this is the question to tell that story on.

Public 2026 Node.js candidate report | reported Q13

How do you safely deploy without downtime?

I use rolling, blue-green, or canary deployment depending on the infra. Before an old instance is terminated, it stops receiving new traffic while in-flight requests finish through graceful shutdown. I rely on readiness and liveness checks, backward-compatible API/DB changes, and monitoring during rollout. For anything risky I prefer canary — a small percentage of traffic first — and stop or roll back immediately if metrics degrade.

Make it yours: Name the actual deployment strategy your team uses (Kubernetes rolling update, etc.) and one time a canary caught a problem early.

Coditas | reported Q14

Your application becomes slow after a new deployment. How would you debug it?

I compare the new deployment against the last healthy version — error rate, p95/p99 latency, CPU, memory, event-loop lag, and DB latency. I look at what actually changed in that deploy — code or config — and use tracing to pinpoint where the extra latency was introduced. If impact is severe, I roll back or flag off the change while I keep investigating, then validate the fix with load testing that mirrors production before calling it done.

Make it yours: This pairs naturally with the API-latency-spike story above — use the same real incident if you have one, told from the deployment angle.

Coditas | reported Q15

How do you identify blocking operations in Node.js?

I look for synchronous filesystem calls, large JSON parse/stringify, CPU-heavy loops, synchronous crypto, and other CPU-bound JS. I correlate CPU usage with event-loop lag and profile to find the exact expensive function. If the work is genuinely CPU-heavy, I move it to Worker Threads, a separate process, or a background worker rather than leaving it on the API's event loop.

Make it yours: If you've profiled with clinic.js or --prof and found a specific offending function, name it — that specificity is what makes this answer land.

Pearson | reported Q16

What Redis cache-eviction policies do you know, and how would you choose one?

It depends on whether Redis should be free to evict any key or only ones with an expiry set. Broadly there's LRU-style eviction (recency-based) and LFU-style eviction (frequency-based), plus policies restricted to expiring keys only. I'd pick LFU for frequently-hit hot data and LRU for recency-driven access patterns, and I always set sensible TTLs rather than leaning on eviction alone to manage memory.

Make it yours: If your team uses Redis for session or catalog caching, mention which policy is actually configured and why.

Pearson | reported Q17

How can Redis be used as a Pub/Sub manager?

Publishers send messages to channels and subscribers receive them — useful for real-time notifications or communication between app instances. But standard Redis Pub/Sub isn't a durable queue: if a subscriber is disconnected, it simply misses those messages. When I need durable delivery, replay, or stronger guarantees, I reach for Redis Streams or a dedicated system like Kafka instead.

Make it yours: If you've used Pub/Sub for something like live order-status updates, mention it and the durability trade-off you accepted.

Pearson | reported Q18

What happens to a Worker Thread under high CPU load, and how would you terminate it?

First I check whether the worker is doing expected CPU-heavy work or is actually stuck, by monitoring its execution time and the app's overall health. Node's Worker API supports explicit termination when needed. In production I also put boundaries around CPU-heavy jobs — timeouts, cancellation where supported, queue limits, and a bounded worker-pool size — because spinning up unlimited workers just increases CPU contention and slows the whole service down.

Make it yours: If you've sized a worker pool for a background job (report generation, image processing), give the actual pool size you landed on.

Cognizant | reported Q19

How do you identify a failure or performance issue in production?

I start with observability — logs, metrics, traces — and figure out the blast radius: request rate, error rate, p95/p99 latency, CPU, memory, event-loop lag, DB latency, downstream failures. I correlate the timing against recent deployments, config changes, and traffic patterns. If there's customer impact, I mitigate first, then find the root cause, then validate the permanent fix before closing it out.

Make it yours: This is the umbrella question — keep one real incident ready that you can retell at three depths: 30 seconds, 90 seconds, and a full walkthrough if they dig in.

Klarna | reported Q20

There's a memory leak in this code — find it.

I look for resources that outlive the component or operation that created them. In React that's usually event listeners, intervals, subscriptions, or async work that never got cleaned up. I check the useEffect cleanup function and whether a closure is holding onto something large. I reproduce it through repeated mount/unmount cycles and use the browser's memory tools to confirm retained objects stop growing once the fix is in.

Make it yours: Mention any live-coding memory-leak exercises you've practiced — interviewers ask this as a whiteboard/live-code task, not just a verbal one.

Remaining questions (14)

Laerdal Medical Corp. | reported Q1

What's the difference between `process.exit()` and a graceful shutdown?

`process.exit()` terminates the process immediately and can cut off pending async work mid-flight. In production I generally prefer graceful shutdown — stop new traffic, let or safely cancel existing work, close resources, then exit. I'd reserve immediate `process.exit()` for situations where continuing is actually unsafe.

Coditas | reported Q2

How would you optimize database queries in production?

I find the slow query via DB monitoring or app tracing, then check its execution plan, indexes, joins, filters, and how much data it returns. I avoid pulling unnecessary columns or rows and make sure indexes actually match the query pattern. I also check connection-pool utilization and transaction duration, then benchmark the optimized query and monitor it under real load.

Coditas | reported Q3

When would you use Redis caching, and what problems does it solve?

When data is expensive to fetch or compute and can tolerate the consistency trade-offs of caching. It cuts DB load and improves latency. I'm careful about TTL and invalidation, and I think through cache stampede, stale data, memory limits, and what happens if Redis itself goes down. I don't cache everything — invalidation complexity has a real cost.

Coditas | reported Q4

How do you handle concurrency issues in backend systems?

I identify the shared state and the consistency requirement first. Depending on the case, I reach for DB transactions, optimistic locking, atomic updates, distributed locks, or idempotency keys. I never rely on an in-memory Node variable alone, since multiple app instances run concurrently. I also test concurrent requests explicitly — most race conditions never show up in normal single-user testing.

Pearson | reported Q5

How would you use Redis hashes for key management?

Hashes fit when several related fields belong to one logical object — instead of many separate keys, I keep related fields under one hash key, which simplifies updates and retrieval. I still weigh memory usage, TTL needs, and access patterns, since Redis data modeling should follow how the app actually reads and writes that data.

Pearson | reported Q6

Explain Atomics in Node.js.

Atomics provide synchronization primitives for shared memory, mainly when Worker Threads use a SharedArrayBuffer — atomic reads, writes, and compare-and-exchange without races on those specific operations. I'd only reach for them when shared-memory concurrency is genuinely needed; for most backend workloads, message passing between workers is simpler and safer.

Cognizant | reported Q7

What are the advantages and disadvantages of using Redis?

Advantages: very fast in-memory access, great for caching, distributed coordination, some messaging patterns. Disadvantages: extra infrastructure, memory cost, cache-invalidation complexity, stale-data risk, and a new dependency that can fail. I design so a Redis outage degrades the service gracefully rather than taking it down entirely, where the business allows it.

Cognizant | reported Q8

How would you optimize a React application?

Profile before optimizing. On React: unnecessary renders, component boundaries, expensive calculations, large lists. I use virtualization for big lists, code-splitting/lazy-loading for bundle size, and memoization only where profiling proves it helps. I also check API latency, network payload size, and browser main-thread work — not every frontend slowdown is a React rendering problem.

Cognizant | reported Q9

Explain Node.js architecture.

Application JS runs through V8, while libuv handles asynchronous I/O, and the event loop coordinates callbacks and async operations. "Single-threaded JS execution" doesn't mean the whole runtime is single-threaded — libuv and the OS handle work outside the JS thread. For CPU-heavy JS specifically, I'd reach for Worker Threads or separate processes, since blocking the main event loop directly hits request latency.

Klarna | reported Q10

What does React.memo do?

It lets React skip re-rendering a component when its props are considered unchanged, using a shallow comparison by default — so if an object or function prop is recreated on every parent render, the child still re-renders anyway. I use it once profiling shows unnecessary renders and the component actually benefits; I

don't add it everywhere, since the comparison itself has a cost.

Glance | reported Q11

Explain the basics of a JavaScript memory leak.

It happens when objects that are no longer logically needed stay reachable, so the garbage collector can't clean them up. Common causes: global references, closures accidentally holding onto something large, event listeners that were never removed, timers, and growing caches. I diagnose by reproducing the behavior, taking memory snapshots, and comparing retained objects over time.

Glance | reported Q12

Explain Pub/Sub in JavaScript.

Publishers emit events without depending directly on subscribers; subscribers register handlers for specific event types. In production, lifecycle management matters most — if subscriptions are created repeatedly and never removed, that causes both duplicate processing and memory leaks. Subscription cleanup deserves as much attention as publishing does.

Walmart | reported Q13

Explain the JavaScript event loop.

It coordinates synchronous JS execution with queued asynchronous callbacks. In a browser, long-running JS on the main thread blocks rendering and user interaction. In Node, long synchronous or CPU-heavy JS blocks the event loop and delays I/O callbacks. When debugging performance, I look for long-running synchronous work rather than assuming async APIs automatically make the app non-blocking.

Singtel | reported Q14

How would you approach performance in a React + Redux application?

React Profiler first, to see which components render and why. With Redux, I check whether selectors are pulling in more state than components actually need — state placement, selector memoization, unnecessary parent re-renders, expensive calculations. For large datasets, virtualization or server-side pagination. And I always re-measure afterward instead of assuming memoization alone fixed it.
